Q1(a) Explain Natural Language Processing (NLP) and distinguish between the rationalist and empiricist approaches to NLP.

Answer:

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to process, understand, analyze, and generate human language.

Human language is complex because the same word or sentence can have different meanings depending on the context.

Applications of NLP:

- Machine translation
- Chatbots
- Sentiment analysis
- Speech processing
- Question answering
- Information retrieval
- Text classification

Example:

Consider:

> "The bank is near the river."

The NLP system has to understand that the word "bank" refers to the side of a river and not a financial institution.

Rationalist Approach

The rationalist approach uses explicitly defined linguistic rules and knowledge.

The rules are generally created by linguists or experts.

Example:

A grammar rule may be:

$$\text{Sentence} \rightarrow \text{Noun Phrase} + \text{Verb Phrase}$$

For example:

> "The student reads."

can be analyzed using predefined grammar rules.

Characteristics:

- Rule-based
- Expert knowledge is important
- Linguistic rules are explicitly represented
- Does not depend completely on large datasets

Empiricist Approach

The empiricist approach learns language patterns from observed data or corpora.

Instead of manually defining every rule, the system learns statistical or machine-learning patterns from examples.

Example:

Suppose a corpus contains:

> I like NLP.
> I like AI.
> I like machine learning.

The system can learn that "I like" frequently occurs before another word.

Characteristics:

- Data-driven
- Learns from examples
- Uses statistical or machine-learning methods
- Requires language data or corpora

Difference

Rationalist	**Empiricist**
Uses explicit rules	Learns from data
Expert-driven	Data-driven
Rules are manually represented	Patterns are learned
Example: grammar rules	Example: statistical language model

Conclusion:

NLP can use linguistic knowledge as well as data-driven learning to process natural language.

Q1(b) Describe the need of lexical, syntactic, and semantic levels in language processing using suitable examples.

Answer:

NLP needs different levels of analysis because understanding language involves more than simply identifying individual words.

The three important levels are:

1. Lexical level
2. Syntactic level
3. Semantic level

1. Lexical Level

The lexical level deals with words and their basic properties.

It identifies:

- Words
- Tokens
- Morphemes
- Word forms

Example:

Sentence:

> "The student reads a book."

Tokens can be identified as:

The — student — reads — a — book

The lexical level tells us what the individual words are.

2. Syntactic Level

The syntactic level deals with how words are arranged to form phrases and sentences.

It checks grammatical relationships and sentence structure.

Example:

Correct:

> "I went to the market."

Incorrect:

> "Went the I market to."

The words are present, but the second sentence does not follow normal syntactic structure.
Another example:

> "She is going to the market."

is grammatically valid.
But:

> "She are going to the market."

has a subject–verb agreement problem.

3. Semantic Level

Semantic analysis deals with meaning.
It tries to determine what the words and complete sentence mean in context.
Example:

> "Colorless green ideas sleep furiously."

The sentence has a syntactically recognizable structure, but its meaning is semantically unusual.
Another example:

> "The bank is closed."

The word "bank" can have different meanings, but context helps determine the intended meaning.

Why are these levels needed?

$$\boxed{\text{Lexical} \rightarrow \text{Syntactic} \rightarrow \text{Semantic}}$$

- Lexical analysis identifies the words.
- Syntactic analysis determines their grammatical structure.
- Semantic analysis determines their meaning.

Therefore, all three levels are necessary for effective language understanding.

Q1(c) Explain the Byte-Pair Encoding (BPE) technique for subword tokenization with a suitable example.

Answer:

Byte-Pair Encoding (BPE) is a subword tokenization technique used to divide words into smaller units called subwords.

Instead of representing every complete word separately, BPE learns frequently occurring character or subword combinations and merges them.

Why BPE is useful

Suppose a system encounters a rare word that was not present during training.

Instead of treating the complete word as unknown, BPE can break it into smaller known subwords.

Steps of BPE

1. Start with characters or small units.
2. Count adjacent symbol pairs.
3. Find the most frequent pair.
4. Merge that pair.
5. Repeat the process for frequent pairs.
6. Stop when the required vocabulary size is obtained.

Example

Consider the words:

new
new
renew
reset
renew

Initially, words can be represented using smaller character units.

For example:

$$new \rightarrow n\ e\ w$$

If a pair such as

$$n + e$$

occurs frequently, it can be merged:

$$ne$$

Further frequent combinations can also be merged.

Eventually, the vocabulary can contain useful subword units such as:

$$new, \quad re, \quad set$$

A rare word can then be represented using available subwords instead of becoming completely unknown.

Advantages of BPE

- Handles rare words.
- Helps with unknown words.
- Learns frequently occurring subword patterns.
- Reduces vocabulary problems.
- Provides a balance between character-level and word-level tokenization.

Key point:

BPE repeatedly merges the most frequent adjacent pair of symbols to create subword units.

Q2(a) Differentiate between a word and a morpheme. Explain free and bound morphemes with suitable examples.

Answer:

A word is a complete linguistic unit that can generally stand independently in a sentence.

A morpheme is the smallest meaningful unit of a language.

A word can contain one or more morphemes.

Example 1

$$dog$$

Here:

- Word = dog
- Morpheme = dog

So one word contains one free morpheme.

Example 2

$$cats = cat + -s$$

Here:

- Word = cats
- Morphemes = cat + -s

Example 3

$$unbreakable = un - + break + -able$$

So one word contains three morphemes.

Free Morpheme

A free morpheme can stand alone as a word.
Examples:

- cat
- book
- student
- break

Bound Morpheme

A bound morpheme cannot normally stand alone. It must be attached to another morpheme.
Examples:

- -s in cats
- un- in unbreakable
- -able in unbreakable

Difference

Word	**Morpheme**
Complete linguistic unit	Smallest meaningful unit
Can generally stand independently	May be free or bound
Can contain multiple morphemes	Cannot be divided further without losing its specific meaning
Example: cats	Example: cat + -s

Q2(b) Explain rule-based tokenization used in NLP with suitable examples.

Answer:

Rule-based tokenization divides text into tokens using predefined rules.

These rules can be based on:

- Spaces
- Punctuation
- Regular expressions
- Sentence boundaries
- Special patterns

Example 1 – Word Tokenization

Input:

> "I like NLP."

A rule-based tokenizer can produce:

$$I \quad like \quad NLP \quad .$$

The period can be treated as a separate token.

Example 2 – Email

Input:

> "Contact abc@gmail.com."

A suitable rule can recognize:

`abc@gmail.com`

as one email token.

Example 3 – URL

Input:

> "Visit www.example.com"

A rule can identify:

`www.example.com`

as one URL token.

Advantages

- Easy to implement.
- Easy to understand.
- Rules can be customized.
- Useful for structured text such as emails, URLs and dates.

Key point:

Rule-based tokenization uses predefined rules, often with regular expressions, to identify and separate tokens.

Q2(c) Explain the Minimum Edit Distance algorithm. Illustrate insertion, deletion and substitution operations using a suitable pair of words.

Answer:

Minimum Edit Distance (MED) is the minimum number of editing operations required to transform one word into another.

The three basic operations are:

1. **Insertion** – adding a character.
2. **Deletion** – removing a character.
3. **Substitution** – replacing one character with another.

1. Insertion

Example:

$$cat \rightarrow cart$$

The character `r` is inserted.

2. Deletion

Example:

$$cart \rightarrow cat$$

The character `r` is deleted.

3. Substitution

Example:

$$cat \rightarrow cut$$

The character `a` is replaced by `u`.

Example: CAT to CUT

Only one substitution is required:

$$CAT \rightarrow CUT$$

Therefore:

$$\boxed{MED(CAT, CUT) = 1}$$

Dynamic Programming Table

	ϵ	C	U	T
ϵ	0	1	2	3
C	1	0	1	2
A	2	1	1	2
T	3	2	2	1

The bottom-right value gives the minimum edit distance:

$$\boxed{1}$$

General Formula

If the characters are equal:

$$D[i][j] = D[i-1][j-1]$$

If they are different:

$$D[i][j] = 1 + \min \begin{cases} D[i-1][j] & \text{Deletion} \\ D[i][j-1] & \text{Insertion} \\ D[i-1][j-1] & \text{Substitution} \end{cases}$$

Important:

The bottom-right cell of the dynamic programming table gives the final minimum edit distance.

MODULE 2 – N-GRAM LANGUAGE MODELS

Q3(a) Determine the bigram counts and calculate $P(AI|like)$ and $P(NLP|like)$.

Given corpus:

$$< s > \ I \ like \ NLP \ < /s >$$

$$< s > \ I \ like \ AI \ < /s >$$

Step 1: Identify the bigrams

First sentence:

$$< s > \ I$$

$$I \ like$$

$$like \ NLP$$

$$NLP \ < /s >$$

Second sentence:

$$< s > \ I$$

$$I \ like$$

$$like \ AI$$

$$AI \ < /s >$$

Step 2: Count the bigrams

Bigram	Count
$\langle s \rangle, I$	2
$I, like$	2
$like, NLP$	1
$NLP, \langle /s \rangle$	1
$like, AI$	1
$AI, \langle /s \rangle$	1

Step 3: Bigram Probability Formula

$$\boxed{P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}}$$

The word `like` occurs twice as the previous word.

Therefore:

$$P(AI|like) = \frac{C(like, AI)}{C(like)}$$

$$= \frac{1}{2}$$

$$\boxed{P(AI|like) = 0.5}$$

Similarly:

$$P(NLP|like) = \frac{1}{2}$$

$$\boxed{P(NLP|like) = 0.5}$$

Q3(b) A language model assigns probabilities 0.5, 0.25, 0.2 and 0.4 to four successive words. Calculate the perplexity.

Answer:

The perplexity formula is:

$$\boxed{PP(W) = \left(\frac{1}{P(w_1 w_2 \cdots w_N)}\right)^{1/N}}$$

Here:

$$N = 4$$

Therefore:

$$PP = \left(\frac{1}{0.5 \times 0.25 \times 0.2 \times 0.4}\right)^{1/4}$$

First calculate the product:

$$0.5 \times 0.25 \times 0.2 \times 0.4 = 0.01$$

Therefore:

$$PP = (100)^{1/4}$$

$$\boxed{PP \approx 3.16}$$

Interpretation:

Lower perplexity generally indicates lower uncertainty and better predictive performance when comparing models under comparable evaluation conditions.

Q3(c) Calculate the probability of generating the sequence $< s > \ I \ like \ NLP$.

Given:

$$P(I| < s >) = 0.6$$

$$P(like|I) = 0.5$$

$$P(NLP|like) = 0.4$$

The sequence is:

$$< s > \ I \ like \ NLP$$

For a bigram model:

$$P(< s > \ I \ like \ NLP) = P(I| < s >) \times P(like|I) \times P(NLP|like)$$

Substitute:

$$= 0.6 \times 0.5 \times 0.4$$

$$= 0.30 \times 0.4$$

$$\boxed{= 0.12}$$

Therefore:

$$\boxed{P(< s > \ I \ like \ NLP) = 0.12}$$

or

$$\boxed{12\%}$$

Significance:

The probability of a sequence in a bigram model is calculated by multiplying the conditional probabilities of successive word pairs.

Q4(a) Calculate the unigram probability of each word.

Given:

Word	Count
NLP	40
AI	30
ML	20
Data	10

Step 1: Total Count

$$N = 40 + 30 + 20 + 10$$

$$N = 100$$

Step 2: Formula

$$\boxed{P(w) = \frac{C(w)}{N}}$$

Step 3: Calculate

NLP:

$$P(NLP) = \frac{40}{100} = 0.4$$

AI:

$$P(AI) = \frac{30}{100} = 0.3$$

ML:

$$P(ML) = \frac{20}{100} = 0.2$$

Data:

$$P(Data) = \frac{10}{100} = 0.1$$

Final Answer

Word	Probability
NLP	0.4
AI	0.3
ML	0.2
Data	0.1

Check:

$$0.4 + 0.3 + 0.2 + 0.1 = 1$$

Q4(b) A bigram language model has a vocabulary of 8 words. The bigram "data science" occurs 0 times, while "data" occurs 20 times. Using Laplace smoothing, calculate $P(science|data)$.

Answer:

Given:

$$V = 8$$

$$C(data, science) = 0$$

$$C(data) = 20$$

Laplace smoothing is also called Add-1 smoothing.

Formula

$$\boxed{P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}}$$

Substitute:

$$P(science|data) = \frac{0 + 1}{20 + 8}$$

$$= \frac{1}{28}$$

$$\boxed{P(science|data) \approx 0.0357}$$

Interpretation:

Without smoothing:

$$P(science|data) = 0$$

because the bigram was never observed.

Laplace smoothing assigns a small non-zero probability to the unseen bigram.

Q4(c) A bigram "natural language" occurs 3 times, and "natural" occurs 30 times. The vocabulary size is 20.

Answer:

Given:

$$C(natural, language) = 3$$

$$C(natural) = 30$$

$$V = 20$$

The question asks for Add-1 and Add-0.5 smoothing.

(a) General Add-k Formula

$$\boxed{P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + k}{C(w_{i-1}) + kV}}$$

(b) Add-1 Smoothing

For Add-1:

$$k = 1$$

Therefore:

$$P(language|natural) = \frac{3 + 1}{30 + (1)(20)}$$

$$= \frac{4}{50}$$

$$\boxed{P(language|natural) = 0.08}$$

(c) Add-0.5 Smoothing

For Add-0.5:

$$k = 0.5$$

Therefore:

$$P(language|natural) = \frac{3 + 0.5}{30 + (0.5)(20)}$$

$$= \frac{3.5}{40}$$

$$\boxed{P(language|natural) = 0.0875}$$

(d) Comparison

The original unsmoothed probability is:

$$P(language|natural) = \frac{3}{30}$$

$$= 0.1$$

Comparison:

Method	Probability
Original	0.1000
Add-1	0.0800
Add-0.5	0.0875

Add-0.5 gives a value closer to the original probability because it applies less smoothing.

Important:

The correct probability depends on the value of k specified in the question.

$$\boxed{\text{Add-1} \Rightarrow k = 1}$$

$$\boxed{\text{Add-0.5} \Rightarrow k = 0.5}$$

Both answers are correct for their respective smoothing values.

QUICK REVISION FORMULAS

Module 1

BPE:

$$\boxed{\text{Find frequent pair} \rightarrow \text{Merge} \rightarrow \text{Repeat}}$$

Minimum Edit Distance:

If characters are equal:

$$D[i][j] = D[i-1][j-1]$$

If different:

$$\boxed{D[i][j] = 1 + \min \begin{cases} D[i-1][j] & \text{Deletion} \\ D[i][j-1] & \text{Insertion} \\ D[i-1][j-1] & \text{Substitution} \end{cases}}$$

Remember:

$$\boxed{\text{Insertion} + \text{Deletion} + \text{Substitution}}$$

The bottom-right cell gives the final MED.

Module 2

Unigram Probability:

$$\boxed{P(w) = \frac{C(w)}{N}}$$

Bigram Probability:

$$\boxed{P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}}$$

Bigram Sentence Probability:

$$\boxed{P(w_1, \ldots, w_n) = \prod_i P(w_i|w_{i-1})}$$

Perplexity:

$$\boxed{PP(W) = \left(\frac{1}{P(w_1 w_2 \cdots w_N)}\right)^{1/N}}$$

Laplace / Add-1 Smoothing:

$$\boxed{P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}}$$

Add-k Smoothing:

$$\boxed{P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + k}{C(w_{i-1}) + kV}}$$

Remember:

$$\boxed{\text{Add-1} \Rightarrow k = 1}$$

$$\boxed{\text{Add-0.5} \Rightarrow k = 0.5}$$